

INSTITUTO FEDERAL DA BAHIA
BACHARELADO EM SISTEMAS DE INFORMAÇÃO

ANDREI SILVA DOS SANTOS
ISAAC LIMA MOREIRA
KAUAN CAMPOS DE JESUS
WERNER LUDWIG MACHADO DE ALMEIDA

JAVA SWING

FEIRA DE SANTANA
2026

ANDREI SILVA DOS SANTOS
ISAAC LIMA MOREIRA
KAUAN CAMPOS DE JESUS
WERNER LUDWIG MACHADO DE ALMEIDA

JAVA SWING

Pesquisa avaliativa em grupo apresentada
como critério avaliativo da disciplina de
Linguagem de Programação II do terceiro
semestre do curso Bacharelado em
Sistemas de Informação no Instituto
Federal da Bahia

Orientador: Prof. Dr. Cleber Jorge Lira de
Santana

FEIRA DE SANTANA

2026

Sumário

1 INTRODUÇÃO.....	1
2 JAVA SWING E SUAS CARACTERÍSTICAS.....	1
3.1 Layout Managers.....	4
5 APLICAÇÃO BASEADA EM EVENTOS.....	7
6 EVENT-DRIVEN PROGRAMMING.....	8
7 TRATAMENTO DE EVENTOS.....	9
8 SDI E MDI.....	10
8.1 Single Document Interface (SDI).....	11
8.1.1 Conceitos.....	11
8.1.2 Funcionamento.....	12
8.1.3 Vantagens.....	13
8.1.4 Desvantagens.....	13
8.1.5 Exemplos de utilização.....	14
9 Multiple Document Interface (MDI).....	14
9.1.1 Conceitos.....	14
9.1.2 Funcionamento.....	15
9.1.3 Vantagens.....	16
9.1.4 Desvantagens.....	17
9.1.5 Exemplo de utilização.....	17
10 SDI ou MDI?.....	18

1 INTRODUÇÃO

O Java Swing disponibiliza diversos componentes gráficos e recursos que permitem ao desenvolvedor construir interfaces para diferentes tipos de aplicações. Além da organização visual dos componentes, as aplicações desenvolvidas com Swing utilizam um modelo baseado em eventos, no qual determinadas ações realizadas pelo usuário, como clicar em um botão ou interagir com um campo de texto, podem desencadear a execução de determinados trechos do programa. Esse mecanismo estabelece uma relação entre os componentes gráficos, os eventos produzidos e os objetos responsáveis por tratá-los, sendo fundamental para o funcionamento de aplicações interativas. (ALURA, 2022).

Dessa forma, esta pesquisa tem como objetivo apresentar os principais conceitos relacionados ao Java Swing e ao desenvolvimento de interfaces gráficas em Java. Serão abordados o conceito de Swing, suas principais características, as diferenças básicas entre aplicações de console e aplicações com interface gráfica, a organização de aplicações baseadas em eventos, o conceito de *Event-Driven Programming* e o tratamento de eventos em componentes gráficos. A compreensão desses conceitos fornece uma base teórica para o desenvolvimento e a implementação de aplicações gráficas utilizando a linguagem Java.

2 JAVA SWING E SUAS CARACTERÍSTICAS

O Java Swing é um conjunto de componentes e recursos da linguagem Java destinado ao desenvolvimento de interfaces gráficas de usuário (*Graphical User Interface* ou *GUI*). Ele faz parte das *Java Foundation Classes (JFC)*, conjunto de tecnologias disponibilizadas pela plataforma Java para a construção de aplicações com interfaces gráficas. O Swing fornece componentes como janelas, botões, caixas de texto, menus, tabelas e painéis, permitindo que o desenvolvedor construa interfaces para aplicações desktop. Segundo a documentação da Oracle, o Swing é formado por componentes gráficos escritos inteiramente em Java, oferecendo recursos para a criação de interfaces com diferentes níveis de complexidade. (ORACLE, 2026).

Uma das principais características do Swing é a grande variedade de componentes gráficos disponibilizados pela biblioteca. Entre os componentes mais

utilizados estão o *JFrame*, responsável por representar uma janela principal da aplicação; o *JPanel*, utilizado para organizar outros componentes; o *JButton*, utilizado para representar botões de interação; o *JLabel*, utilizado para apresentar textos ou informações; o *TextField*, destinado à entrada de dados; além de componentes mais elaborados, como *JTable*, *JMenuBar* e *JComboBox*. Essa variedade permite organizar a interface de acordo com as necessidades da aplicação, possibilitando a criação de telas de cadastro, consulta, menus e outras funcionalidades. (ORACLE, 2026; ALURA, 2022).

Outra característica importante é a independência dos componentes em relação ao sistema operacional, uma vez que o Swing utiliza componentes implementados em Java e permite que a aparência e o comportamento da interface sejam controlados pela própria aplicação. O Swing também utiliza o conceito de *Look and Feel*, que permite modificar a aparência dos componentes sem alterar necessariamente sua funcionalidade. Além disso, a biblioteca possui mecanismos de organização dos componentes por meio de gerenciadores de layout, permitindo definir como botões, campos de texto, painéis e outros elementos serão posicionados dentro das janelas. Dessa maneira, o desenvolvedor consegue estruturar interfaces de forma organizada e adaptável. (ORACLE, 2026; BRASIL, Ministério da Educação, 2012).

Por fim, o Swing possui como característica fundamental seu modelo de programação baseado em eventos. Os componentes gráficos podem gerar eventos a partir das ações realizadas pelo usuário, como clicar em um botão, selecionar uma opção, movimentar o mouse ou pressionar uma tecla. Esses eventos podem ser associados a objetos chamados listeners, responsáveis por receber as notificações e executar o código correspondente à ação realizada. Esse modelo permite que a aplicação responda de maneira dinâmica às interações do usuário, sendo um dos fundamentos para o desenvolvimento de aplicações gráficas interativas em Java.

3 COMPONENTES SWING

O Swing disponibiliza uma ampla variedade de componentes gráficos que podem ser combinados para construir as diferentes telas de uma aplicação. Cada componente possui uma finalidade específica, desde elementos simples para

exibição de informações, como *JLabel*, até componentes destinados à entrada, seleção e apresentação de dados, como *TextField*, *ComboBox* e *JTable*. Esses componentes podem ser adicionados a outros componentes, organizados por meio de *Layout Managers* e configurados para responder às ações do usuário. (ORACLE, 2026; BRASIL, Ministério da Educação, 2012).

Para facilitar a compreensão dos principais componentes disponibilizados pelo Java Swing, a Tabela 1 apresenta uma síntese de suas respectivas finalidades e exemplos de utilização em aplicações gráficas.

Tabela 1 - Tabela de Componentes.

COMPONENTE	FINALIDADE	EXEMPLO
JFrame	Representa uma janela principal da aplicação	Janela principal de um sistema de cadastro.
JInternalFrame	Representa uma janela interna que pode ser exibida dentro de um JDesktopPane.	Tela de cadastro de clientes dentro da janela principal.
JPanel	Área utilizada para agrupar e organizar outros componentes.	Separar campos de cadastro, botões e tabelas em diferentes áreas da tela.
JMenuBar	Barra que contém os menus da aplicação.	Barra superior contendo "Arquivo", "Editar" e "Ajuda".
JMenu	Representa um menu que pode conter itens ou outros menus.	Menu "Cadastro" contendo opções como "Cliente" e "Produto".
JMenuItem	Representa uma opção selecionável dentro de um JMenu.	Item "Cadastrar Cliente" dentro do menu "Cadastro".
JLabel	Exibe textos, imagens ou outras informações que não exigem edição direta pelo usuário.	Rótulos como "Nome:", "CPF:" e "Endereço:".
TextField	Permite a entrada de uma única linha de texto.	Campo para informar o nome de um cliente.
PasswordField	Permite a entrada de uma senha, ocultando visualmente os caracteres digitados.	Campo de senha em uma tela de login.

JTextArea	Permite inserir e exibir múltiplas linhas de texto.	Campo para observações ou descrição de um produto.
JButton	Representa um botão que pode executar uma ação quando acionado.	Botões "Cadastrar", "Excluir", "Editar" e "Cancelar".
JComboBox	Apresenta uma lista suspensa para seleção de uma opção.	Seleção do estado civil ou estado de um endereço.
JRadioButton	Permite selecionar uma opção entre alternativas, normalmente utilizado em conjunto com ButtonGroup.	Escolha entre "Masculino" e "Feminino", quando esse conjunto de opções for apropriado ao sistema.
JCheckBox	Permite selecionar ou desmarcar uma opção independentemente de outras opções.	Seleção de serviços adicionais ou preferências.
JTable	Apresenta dados organizados em linhas e colunas.	Listagem de clientes, produtos ou funcionários.
JScrollPane	Fornecer barras de rolagem para componentes cujo conteúdo pode ultrapassar sua área disponível.	Permitir rolagem de uma JTable ou JTextArea.
JOptionPane	Fornecer caixas de diálogo prontas para mensagens, confirmações e entrada de dados.	Exibir "Cadastro realizado com sucesso!" ou solicitar confirmação antes de excluir um registro.

Fonte: Autoria própria.

A Tabela 1 evidencia que o Java Swing disponibiliza componentes com diferentes funções, permitindo estruturar desde interfaces simples até aplicações gráficas mais completas. A combinação desses componentes possibilita criar telas para entrada e apresentação de dados, navegação por menus, seleção de opções e interação com o usuário. Assim, o conhecimento da finalidade de cada componente é fundamental para que o desenvolvedor possa escolher os recursos mais adequados à construção e organização da interface gráfica da aplicação.

3.1 Layout Managers.

Além dos componentes gráficos, o Swing utiliza *Layout Managers* (gerenciadores de layout) para determinar como os componentes serão posicionados

e dimensionados dentro de um contêiner, como um *JPanel* ou *JFrame*. Em vez de definir manualmente as coordenadas de cada componente, o desenvolvedor pode utilizar um gerenciador de layout para organizar automaticamente os elementos da interface. Essa abordagem facilita a construção de interfaces que podem se adaptar às alterações no tamanho da janela. Entre os principais gerenciadores estão *FlowLayout*, *BorderLayout* e *GridLayout*. (ORACLE, 2026).

FlowLayout organiza os componentes sequencialmente, normalmente da esquerda para a direita, colocando-os em uma mesma linha enquanto houver espaço disponível. Quando não houver espaço suficiente, os componentes podem ser transferidos para a próxima linha. É um layout simples e bastante utilizado quando se deseja organizar pequenos grupos de componentes de maneira sequencial, como uma sequência de botões. (ORACLE, 2026).

BorderLayout divide o espaço disponível em cinco regiões: *NORTH*, *SOUTH*, *EAST*, *WEST* e *CENTER*. Um componente pode ser colocado em cada uma dessas regiões, sendo a região central normalmente utilizada para ocupar o espaço restante. Esse gerenciador é particularmente útil para estruturar janelas que possuem diferentes áreas, como uma barra superior, uma área central de conteúdo e uma área inferior para comandos. (ORACLE, 2026).

GridLayout organiza os componentes em uma grade composta por linhas e colunas, fazendo com que as células tenham dimensões uniformes. É adequado para situações em que os componentes precisam seguir uma estrutura matricial, como determinados formulários, teclados virtuais ou conjuntos de botões. Por exemplo, um *GridLayout(2, 3)* pode organizar seis componentes em duas linhas e três colunas. (ORACLE, 2026).

Além dos componentes gráficos, o Swing também oferece os chamados *Layout Managers*, que são responsáveis por organizar e posicionar os componentes dentro das janelas e painéis. A tabela 2 apresenta três dos principais layouts utilizados em Java: o *FlowLayout*, o *BorderLayout* e o *GridLayout*, mostrando de forma simplificada como cada um organiza os elementos da interface.

Tabela 2 - Tabela de Layouts.

LAYOUT	ORGANIZAÇÃO	EXEMPLO
FlowLayout	Botão 1, Botão 2, Botão 3	Grupo de botões
BorderLayout	NORTH, WEST, CENTER, EAST, SOUTH.	Estrutura geral de uma janela
GridLayout	<pre>[1][2][3] [4][5][6]</pre>	Grade de botões ou campos

Fonte: Autoria própria.

Podemos observar que cada layout possui uma forma diferente de organizar os componentes. O FlowLayout coloca os elementos em sequência, sendo útil, por exemplo, para organizar botões. O BorderLayout divide a janela em cinco regiões, sendo bastante utilizado para estruturar diferentes áreas de uma aplicação. Já o GridLayout organiza os componentes em linhas e colunas, formando uma grade. Portanto, a escolha do layout depende da forma como queremos estruturar os elementos dentro da interface.

4 APLICAÇÕES DE CONSOLE X APLICAÇÕES COM INTERFACE GRÁFICA

As aplicações de console são programas cuja interação com o usuário ocorre principalmente por meio de texto, geralmente utilizando um terminal ou linha de comandos. Nesse modelo, o programa apresenta informações na tela e recebe dados por meio de comandos ou entradas fornecidas pelo usuário, seguindo normalmente uma sequência de execução previamente definida. Esse tipo de aplicação é característico de programas mais simples ou de ferramentas voltadas à administração e automação de sistemas, nas quais uma interface visual nem sempre é necessária. (BRASIL, Ministério da Educação, 2012).

Em contrapartida, as aplicações com interface gráfica utilizam elementos visuais para permitir a interação com o usuário, como janelas, botões, menus, campos de texto, caixas de seleção e tabelas. Em vez de depender exclusivamente da digitação de comandos, o usuário pode realizar ações diretamente sobre esses componentes. No Java, o Swing fornece diversos desses elementos e permite a construção de interfaces gráficas para aplicações desktop. Além disso, essas

aplicações normalmente utilizam um modelo orientado a eventos, no qual ações do usuário, como clicar em um botão ou selecionar uma opção, geram eventos que podem ser tratados pelo programa. (ALURA, 2022).

A principal diferença, portanto, está na forma de interação e no fluxo de execução. Em uma aplicação de console, a interação ocorre predominantemente por meio de texto e tende a seguir uma sequência de entradas e saídas, enquanto uma aplicação gráfica permanece preparada para responder a diferentes ações realizadas pelo usuário sobre a interface. Isso não significa que uma abordagem seja necessariamente superior à outra, pois a escolha depende dos objetivos da aplicação. Para sistemas que exigem maior interação visual e facilidade de utilização, uma interface gráfica pode ser mais adequada, enquanto aplicações de console podem ser mais simples e eficientes para determinadas tarefas técnicas e automatizadas.

5 APLICAÇÃO BASEADA EM EVENTOS

Uma aplicação baseada em eventos é organizada de maneira diferente de um programa que simplesmente executa suas instruções de forma sequencial. Nesse modelo, a aplicação é preparada para aguardar determinadas ações ou ocorrências, denominadas eventos, que podem ser geradas pelo usuário ou pelo próprio sistema. Entre os exemplos de eventos estão o clique em um botão, a seleção de uma opção, o pressionamento de uma tecla ou uma alteração realizada em determinado componente da interface. Dessa forma, o funcionamento da aplicação está relacionado à ocorrência desses eventos e às ações que devem ser executadas em resposta a eles.

No Java Swing, essa organização envolve principalmente três elementos: os componentes gráficos, os eventos e os objetos responsáveis pelo tratamento desses eventos. Os componentes, como *JButton*, *JTextField* e *JComboBox*, fazem parte da interface e podem produzir diferentes tipos de eventos. Quando uma determinada ação ocorre, um objeto *listener* pode receber a notificação desse evento e executar o código correspondente. Assim, o comportamento da aplicação é definido pela associação entre os componentes da interface e os mecanismos responsáveis por responder às interações do usuário.

De maneira simplificada, o funcionamento ocorre em quatro etapas: o componente da interface gera um evento, o *listener* recebe essa ocorrência e executa a ação correspondente. Por exemplo, em uma tela de cadastro, um *JButton* pode representar o botão "Cadastrar". Quando o usuário clica nesse botão, é gerado um evento de ação, que pode ser recebido por um *ActionListener*. O código associado ao *listener* pode então realizar operações como validar os dados preenchidos, armazenar as informações e apresentar uma mensagem ao usuário. Esse mecanismo permite separar a interação realizada na interface da lógica executada em resposta a ela.

Portanto, a organização baseada em eventos torna a aplicação preparada para responder continuamente às interações realizadas durante sua execução, em vez de depender exclusivamente de uma sequência fixa de comandos. No Swing, esse modelo é fundamental para transformar os componentes gráficos em elementos interativos, permitindo que cada botão, campo, menu ou outro componente execute comportamentos específicos quando determinados eventos ocorrerem. (BRASIL, Ministério da Educação, 2012).

6 EVENT-DRIVEN PROGRAMMING

A *Event-Driven Programming*, ou Programação Orientada a Eventos, é um modelo de programação no qual o fluxo de execução de uma aplicação é determinado pela ocorrência de eventos. Em vez de executar todas as instruções seguindo necessariamente uma sequência previamente estabelecida, o programa permanece preparado para identificar e responder a diferentes acontecimentos durante sua execução. Esses acontecimentos podem ser provocados pelo usuário, como um clique do mouse ou o pressionamento de uma tecla, ou pelo próprio sistema e pelos componentes da aplicação.

Nesse modelo, os eventos funcionam como estímulos que desencadeiam determinadas ações dentro do programa. Para isso, componentes da interface podem gerar eventos, enquanto mecanismos específicos, como os *event listeners*, ficam responsáveis por receber esses eventos e executar o comportamento correspondente. No Java Swing, por exemplo, quando o usuário clica em um *JButton*, o componente pode gerar um evento de ação que é encaminhado ao

ActionListener associado a ele. O código implementado nesse listener determina o que a aplicação deverá fazer após o clique.

A programação orientada a eventos é especialmente importante no desenvolvimento de interfaces gráficas porque o usuário não necessariamente realiza as ações em uma ordem previsível. Em uma tela de cadastro, por exemplo, o usuário pode preencher diferentes campos, selecionar opções, clicar em botões ou cancelar uma operação em momentos distintos. A aplicação precisa, portanto, permanecer disponível para responder a essas interações conforme elas acontecem. Assim, o modelo orientado a eventos permite desenvolver sistemas mais interativos, nos quais o comportamento da aplicação é determinado pelas ações realizadas durante sua utilização.

No Java Swing, esse conceito constitui uma das bases para o desenvolvimento de aplicações gráficas interativas. Os componentes da interface são associados aos mecanismos de tratamento de eventos, permitindo que cada interação produza uma resposta específica. Dessa forma, a programação orientada a eventos estabelece uma relação entre evento, componente, mecanismo de tratamento e ação, possibilitando que a interface gráfica responda dinamicamente ao comportamento do usuário. (ORACLE, 2026; BRASIL, Ministério da Educação, 2012).

7 TRATAMENTO DE EVENTOS

O tratamento de eventos em componentes gráficos consiste em definir como a aplicação deverá reagir quando determinada interação ou ocorrência acontecer. Em uma interface gráfica, ações como clicar em um botão, selecionar uma opção, pressionar uma tecla ou alterar o estado de um componente podem gerar eventos. Para que a aplicação responda a essas ações, é necessário associar os componentes aos mecanismos responsáveis por receber e processar os eventos correspondentes. No Java Swing, essa tarefa é realizada principalmente por meio dos event listeners, que são objetos preparados para receber notificações de eventos produzidos pelos componentes da interface.

O funcionamento desse mecanismo pode ser compreendido a partir da relação entre componente, evento e listener. Um componente gráfico, como um

JButton, pode produzir um evento quando o usuário realiza uma ação sobre ele. Um objeto que implementa a interface ActionListener pode ser registrado nesse botão para receber a notificação. Quando o evento ocorre, o método actionPerformed() do listener é executado, permitindo que o desenvolvedor determine quais instruções deverão ser realizadas em resposta à interação. Dessa maneira, o comportamento do componente é definido pelo código associado ao tratamento do evento.

Por exemplo, em uma tela de cadastro, um botão JButton denominado "Cadastrar" pode possuir um ActionListener associado a ele. Quando o usuário clicar nesse botão, o evento será capturado pelo listener e o método actionPerformed() poderá executar operações como verificar se os campos foram preenchidos corretamente, realizar o cadastro dos dados e apresentar uma mensagem de confirmação. O mesmo princípio pode ser aplicado a diferentes componentes e tipos de eventos, permitindo que cada elemento da interface possua comportamentos específicos de acordo com as ações realizadas pelo usuário.

Assim, o tratamento de eventos constitui uma parte fundamental do desenvolvimento de aplicações gráficas com Swing, pois permite transformar componentes visuais em elementos interativos. O desenvolvedor não precisa apenas criar e posicionar os componentes na interface, mas também definir quais comportamentos serão executados quando o usuário interagir com eles. Dessa forma, a utilização de listeners e dos mecanismos de eventos permite estabelecer a comunicação entre as ações realizadas na interface e a lógica responsável por executar as funcionalidades da aplicação. (BRASIL, Ministério da Educação, 2012).

8 SDI E MDI.

No desenvolvimento de aplicações com interfaces gráficas, a forma como as janelas e os documentos são organizados influencia diretamente a estrutura e a interação do usuário com o sistema. Nesse contexto, destacam-se dois modelos de organização de interfaces: a SDI (Single Document Interface), ou Interface de Documento Único, e a MDI (Multiple Document Interface), ou Interface de Múltiplos Documentos. Esses modelos apresentam diferentes formas de estruturar e organizar as janelas de uma aplicação, sendo utilizados de acordo com as características e

necessidades do sistema. No contexto do Java Swing, o modelo MDI possui recursos específicos, como o `JDesktopPane` e o `JInternalFrame`, destinados à criação e gerenciamento de múltiplas janelas internas dentro de uma aplicação.

Nas próximas subseções, serão apresentados individualmente os conceitos de SDI e MDI, abordando seu funcionamento, principais características, vantagens, desvantagens e exemplos de utilização. Também será apresentada a relação entre o modelo MDI e os componentes disponibilizados pelo Java Swing, permitindo compreender como esse modelo pode ser aplicado na construção de sistemas gráficos.

8.1 Single Document Interface (SDI)

8.1.1 Conceitos

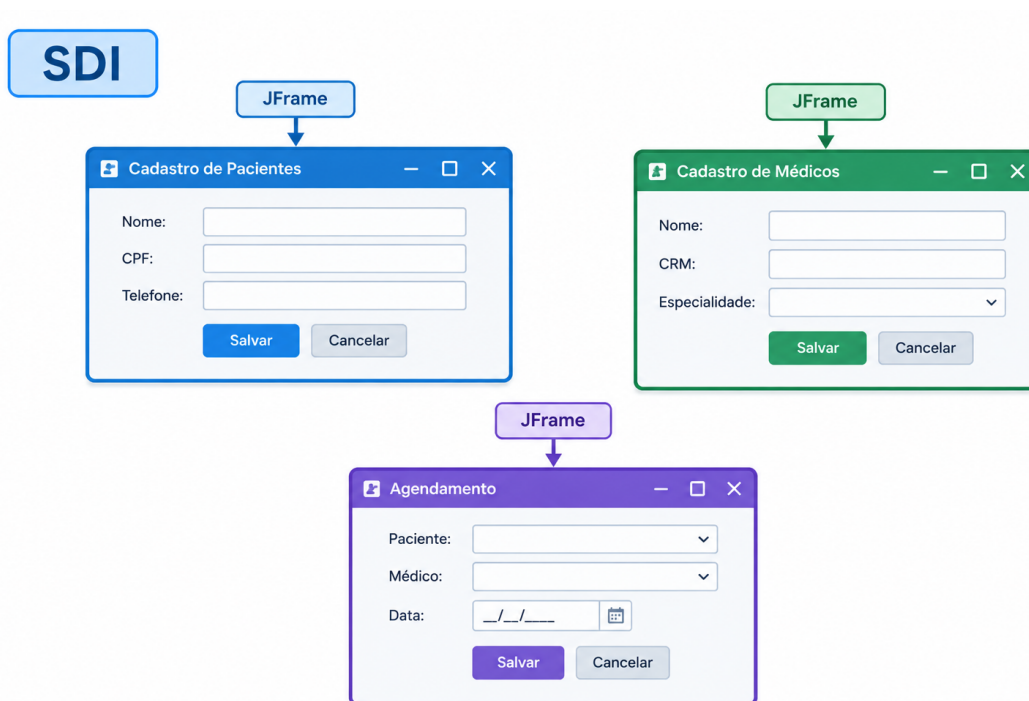
A SDI (Single Document Interface), ou Interface de Documento Único, é um modelo de organização de interfaces gráficas no qual as janelas da aplicação são apresentadas de forma independente. Diferentemente de uma arquitetura em que diversas telas ficam contidas dentro de uma janela principal, no modelo SDI cada janela possui sua própria área de trabalho e pode ser gerenciada separadamente pelo sistema operacional. Dessa forma, as diferentes telas ou documentos da aplicação não ficam restritos a uma área interna de uma janela principal. (DEV MEDIA, 2010; MICROSOFT, 2026).

Nesse modelo, cada janela representa uma unidade independente de interação com o usuário. Isso significa que, quando uma aplicação possui diferentes funcionalidades apresentadas em janelas distintas, essas janelas podem ser movimentadas e organizadas separadamente na área de trabalho. Em aplicações Java Swing, esse comportamento pode ser implementado utilizando componentes de nível superior, como o `JFrame`, que representa uma janela principal ou independente da aplicação. A documentação da Oracle caracteriza o `JFrame` como uma janela de nível superior capaz de possuir título, bordas, controles de janela e componentes gráficos. (ORACLE, 2026).

8.1.2 Funcionamento

O funcionamento do modelo SDI baseia-se na criação de janelas independentes para apresentar as diferentes partes da aplicação. Ao acessar determinada funcionalidade, uma nova janela pode ser apresentada ao usuário sem que ela precise estar contida dentro de uma janela principal. Essas janelas podem ser movimentadas, redimensionadas, minimizadas ou fechadas individualmente, permitindo que o usuário organize sua área de trabalho de acordo com suas necessidades. A independência das janelas é justamente uma das principais características que diferencia o SDI do modelo MDI como demonstra a figura abaixo. (DEVMEDIA, 2010).

Figura 1



Fonte: Autoria própria.

Em Java Swing, uma aplicação baseada nesse modelo pode utilizar diferentes objetos JFrame para representar suas janelas. Por exemplo, um sistema poderia possuir uma janela para o cadastro de clientes e outra para o cadastro de produtos, sendo cada uma apresentada como uma janela independente. Assim, a organização das telas ocorre no próprio ambiente de trabalho do sistema operacional, em vez de todas as telas ficarem restritas à área de uma janela-pai. (ORACLE, 2026).

8.1.3 Vantagens

Uma das principais vantagens do SDI é a simplicidade de organização e implementação. Como as janelas são independentes, não é necessário estabelecer uma hierarquia entre uma janela principal e diversas janelas internas. Essa característica pode facilitar tanto o desenvolvimento quanto a compreensão da interface pelo usuário, principalmente em aplicações que possuem poucas telas ou que não necessitam manter várias funcionalidades abertas simultaneamente. A literatura técnica sobre interfaces gráficas também destaca a simplicidade de implementação e gerenciamento como uma característica positiva do modelo SDI. (DEV MEDIA, 2010).

Outra vantagem está na liberdade de organização das janelas. Como cada tela é independente, o usuário pode posicioná-la em diferentes regiões da área de trabalho e alternar entre as aplicações ou janelas utilizando os recursos disponibilizados pelo sistema operacional. Esse comportamento pode ser interessante quando o usuário precisa consultar simultaneamente informações de diferentes telas ou quando as funcionalidades não precisam compartilhar uma mesma área de trabalho interna.

8.1.4 Desvantagens

Apesar de sua simplicidade, o modelo SDI pode apresentar dificuldades de organização quando uma aplicação possui muitas telas. Como cada janela é independente, a abertura de diversas funcionalidades pode resultar em uma grande quantidade de janelas espalhadas pela área de trabalho. Consequentemente, o usuário pode ter maior dificuldade para identificar, localizar e alternar entre as telas que estão abertas. Essa característica é apontada como uma das questões associadas à utilização de múltiplas janelas independentes. (DEV MEDIA, 2010).

Outra desvantagem está relacionada à ausência de uma área centralizada para o gerenciamento das funcionalidades. Diferentemente do MDI, no qual as janelas internas ficam organizadas dentro de uma janela principal, no SDI as diferentes janelas são tratadas individualmente. Em sistemas de maior complexidade, isso pode tornar a navegação menos organizada e aumentar a quantidade de elementos que precisam ser gerenciados pelo usuário.

8.1.5 Exemplos de utilização

O modelo SDI pode ser utilizado em aplicações nas quais as funcionalidades ou documentos precisam ser apresentados em janelas independentes. Um exemplo são aplicações que permitem abrir diferentes documentos ou recursos separadamente, fazendo com que cada um seja representado por sua própria janela. Nesses casos, o usuário pode alternar entre as janelas utilizando os mecanismos de gerenciamento fornecidos pelo sistema operacional. O modelo também é associado a aplicações desktop nas quais cada formulário é apresentado como uma janela independente. (DEV MEDIA, 2010).

No contexto de um sistema de clínica desenvolvido em Java Swing, por exemplo, o SDI poderia ser utilizado para apresentar funcionalidades como Cadastro de Pacientes, Cadastro de Médicos, Agendamento e Consultas em janelas independentes. Cada funcionalidade poderia ser representada por um JFrame próprio, permitindo que o usuário movesse ou redimensionasse as janelas separadamente. Essa estrutura pode ser adequada para sistemas menores ou para aplicações nas quais a independência entre as telas seja mais importante do que a centralização da interface. (ORACLE, 2026).

Outro exemplo pode ser encontrado em aplicações que trabalham com diferentes tipos de documentos ou ferramentas que precisam ser visualizados separadamente. Nessa situação, cada documento ou funcionalidade pode possuir sua própria janela, permitindo que o usuário mantenha diferentes elementos abertos simultaneamente. Dessa maneira, o SDI oferece uma abordagem direta para aplicações que priorizam a independência das janelas e uma estrutura de interface relativamente simples.

9 Multiple Document Interface (MDI)

9.1.1 Conceitos

A MDI (Multiple Document Interface), ou Interface de Múltiplos Documentos, é um modelo de organização de interfaces gráficas no qual diversas janelas ou documentos são apresentados dentro de uma mesma janela principal da aplicação. Diferentemente do modelo SDI, em que as janelas são independentes, no MDI existe

uma estrutura central que funciona como área de trabalho para as demais telas. Dessa maneira, diferentes funcionalidades podem permanecer abertas simultaneamente e ser organizadas dentro de um mesmo ambiente da aplicação. (DEV MEDIA, 2010; MICROSOFT, 2026).

No Java Swing, o modelo MDI possui componentes específicos para sua implementação. A classe `JDesktopPane` é definida pela documentação da Oracle como um contêiner utilizado para criar uma Multiple Document Interface ou uma área de trabalho virtual. Dentro desse componente são adicionados objetos `JInternalFrame`, que representam as janelas internas da aplicação. Assim, uma estrutura MDI em Swing pode ser formada por uma janela principal, geralmente representada por um `JFrame`, contendo um `JDesktopPane`, no qual são inseridos diferentes `JInternalFrame`. (ORACLE, 2026).

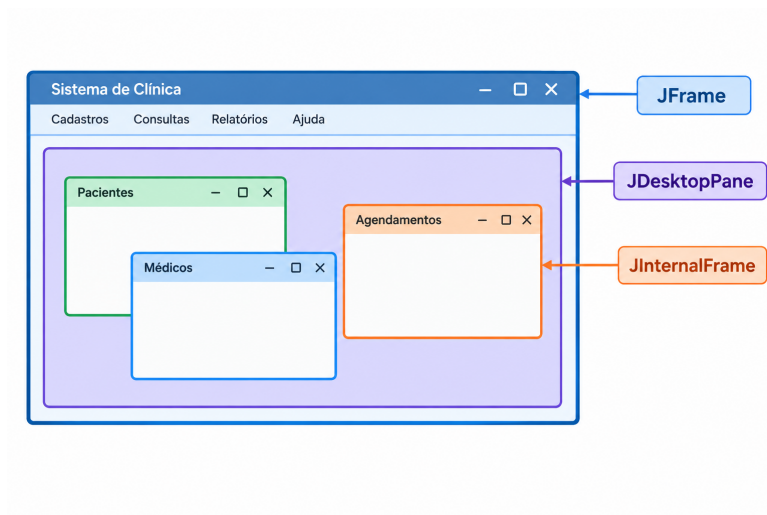
9.1.2 Funcionamento

O funcionamento do MDI ocorre a partir de uma janela principal que concentra a aplicação e fornece uma área na qual as demais janelas podem ser abertas. Dentro dessa área, o usuário pode acessar diferentes funcionalidades sem precisar abrir uma nova janela independente para cada uma delas. As janelas internas podem ser movimentadas, redimensionadas, minimizadas ou fechadas individualmente, permanecendo, entretanto, dentro da área de trabalho definida pela aplicação. (DEV MEDIA, 2010; MICROSOFT, 2026).

No Java Swing, essa estrutura pode ser implementada utilizando um `JFrame` como janela principal, um `JDesktopPane` como área de trabalho e diversos `JInternalFrame` como janelas internas. O `JDesktopPane` permite gerenciar múltiplas janelas internas, inclusive quando elas se sobrepõem, enquanto o `JInternalFrame` fornece recursos semelhantes aos de uma janela convencional, como movimentação, redimensionamento, minimização, fechamento e exibição de título. (ORACLE, 2026).

Uma representação simplificada dessa estrutura pode ser apresentada da seguinte forma:

Figura 1



Fonte: Autoria própria.

9.1.3 Vantagens

Uma das principais vantagens do MDI é a centralização da aplicação. Como as diferentes funcionalidades permanecem dentro de uma janela principal, o usuário consegue acessar e organizar as telas em um mesmo ambiente de trabalho. Menus, barras de ferramentas e outros elementos de navegação também podem ser concentrados na janela principal, proporcionando uma estrutura mais uniforme para o sistema. Essa característica pode ser especialmente útil em aplicações que possuem diversas funcionalidades relacionadas entre si. (DEV MEDIA, 2010; MICROSOFT, 2026).

Outra vantagem está na organização das múltiplas telas. Em vez de distribuir várias janelas independentes pela área de trabalho do sistema operacional, o MDI mantém as janelas internas dentro de uma área controlada pela própria aplicação. No Swing, o **JDesktopPane** também fornece mecanismos para obter, selecionar e gerenciar as diferentes **JInternalFrame** presentes na área de trabalho. (ORACLE, 2026).

9.1.4 Desvantagens

Apesar de proporcionar maior centralização, o MDI também pode apresentar algumas desvantagens. Quando muitas janelas internas são abertas simultaneamente, a área de trabalho pode ficar visualmente sobrecarregada, especialmente quando os `JInternalFrame` se sobrepõem. Nessa situação, o usuário pode ter dificuldade para localizar uma determinada tela ou visualizar simultaneamente as informações necessárias. A própria existência de mecanismos no `JDesktopPane` para controlar a seleção, as camadas e o gerenciamento das janelas demonstra a necessidade de administrar adequadamente essas múltiplas interfaces. (ORACLE, 2026).

Outra desvantagem está relacionada à maior complexidade de implementação e gerenciamento quando comparada a uma aplicação formada apenas por janelas independentes. O desenvolvedor precisa estruturar uma janela principal e administrar as janelas internas que serão apresentadas em seu interior. Em aplicações muito grandes, uma quantidade elevada de telas pode tornar a interface mais difícil de organizar e utilizar, exigindo mecanismos adicionais de navegação e gerenciamento das janelas.

9.1.5 Exemplo de utilização

O MDI pode ser utilizado principalmente em sistemas que possuem diversas funcionalidades relacionadas e que precisam permanecer disponíveis dentro de um mesmo ambiente. Sistemas administrativos, por exemplo, podem utilizar uma janela principal contendo menus para diferentes módulos, enquanto cada funcionalidade é aberta em uma janela interna. Dessa forma, o usuário pode trabalhar com diferentes operações sem abandonar a janela principal do sistema. Esse modelo é particularmente associado a aplicações que precisam apresentar múltiplos documentos ou áreas de trabalho simultaneamente. (DEVMEDIA, 2010; MICROSOFT, 2026).

No contexto de um sistema de clínica desenvolvido em Java Swing, por exemplo, a janela principal poderia conter um `JDesktopPane` responsável por organizar as diferentes funcionalidades do sistema. Ao selecionar a opção Cadastro de Pacientes, seria aberta uma `JInternalFrame` contendo o formulário

correspondente. Da mesma forma, as opções Cadastro de Médicos, Agendamento, Consultas e Relatórios poderiam abrir outras `JInternalFrame` dentro do mesmo `JDesktopPane`. A documentação da Oracle apresenta justamente essa relação entre `JDesktopPane` e `JInternalFrame` como uma forma de construir uma interface de múltiplos documentos. (ORACLE, 2026).

Outro exemplo seria um sistema empresarial que possua módulos de clientes, produtos, vendas, estoque e relatórios. Nesse caso, todas essas funcionalidades poderiam permanecer disponíveis dentro de uma única janela principal, permitindo que o usuário abra diferentes telas internas conforme sua necessidade. Assim, o MDI pode proporcionar uma organização centralizada para sistemas que apresentam grande quantidade de funcionalidades relacionadas, mantendo as diferentes telas dentro de um mesmo ambiente de trabalho.

10 SDI ou MDI?

Os modelos SDI (Single Document Interface) e MDI (Multiple Document Interface) apresentam diferentes formas de organizar as janelas de uma aplicação gráfica. No SDI, cada janela funciona de maneira independente, permitindo que as diferentes funcionalidades sejam apresentadas separadamente na área de trabalho. No MDI, por outro lado, existe uma janela principal que funciona como ambiente de trabalho para as demais janelas internas. Assim, enquanto o SDI prioriza a independência das janelas, o MDI busca centralizar e organizar diferentes funcionalidades dentro de uma mesma aplicação. (DEVMEDIA, 2010).

No contexto do Java Swing, essa diferença também pode ser observada na escolha dos componentes utilizados. Uma aplicação baseada em SDI pode utilizar janelas independentes, como diferentes objetos `JFrame`. Já uma aplicação MDI pode utilizar uma estrutura composta por um `JFrame`, um `JDesktopPane` como área de trabalho e diferentes `JInternalFrame` como janelas internas. A documentação da Oracle define o `JDesktopPane` como um contêiner destinado à criação de uma Multiple Document Interface e indica o uso de `JInternalFrame` dentro desse componente.

A escolha entre SDI e MDI depende das características da aplicação e da forma como se deseja organizar a interação com o usuário. O SDI pode ser

adequado quando as funcionalidades precisam funcionar de maneira independente, enquanto o MDI pode ser interessante em sistemas que possuem diversos módulos relacionados e que devem permanecer concentrados em uma mesma janela principal. Portanto, não existe necessariamente um modelo universalmente superior, mas sim uma escolha de arquitetura de interface de acordo com as necessidades do sistema. A tabela 3 mostra um comparativo direto de

Tabela 3 - SDI x MDI

CARACTERISTICA	SDI	MDI
Estrutura	Janelas independentes	Janelas internas dentro de uma janela principal
Janela Principal	Não é necessária para organizar as demais janelas	É utilizada como ambiente principal da aplicação
Organização	Funcionalidades distribuídas em janelas independentes	Funcionalidades centralizadas em uma área de trabalho
Gerenciamento	Cada janela é gerenciada individualmente	As janelas internas são gerenciadas dentro da aplicação
Java Swing	Pode utilizar JFrame	Utiliza JFrame + JDesktopPane + JInternalFrame
Quantidade de Janelas	Podem existir várias janelas independentes	Podem existir várias janelas internas simultaneamente
Principal Vantagem	Simplicidade e independência das janelas	Centralização e organização das funcionalidades
Principal Desvantagem	Muitas janelas podem ocupar e desorganizar a área de trabalho	Muitas janelas internas podem se sobrepor e dificultar a navegação
Exemplo	Formulários independentes	Sistema de clínica com vários módulos dentro da janela principal

Fonte: Autoria própria.

Referências Bibliográficas

ORACLE. *About the JFC and Swing*. Oracle Java Tutorials. Disponível em: <https://docs.oracle.com/javase/tutorial/uiswing/start/about.html>. Acesso em: 1 set. 2026.

ORACLE. *javax.swing Package*. Java Platform, Standard Edition API Documentation. Disponível em: [Documentação da API javax.swing](#). Acesso em: 1 set. 2026.

ORACLE. *Writing Event Listeners*. Oracle Java Tutorials. Disponível em: <https://docs.oracle.com/javase/tutorial/uiswing/events/>. Acesso em: 1 set. 2026.

ALURA. *Como criar uma interface gráfica em Java com Swing: passo a passo de Java Swing*. Alura, 2022. Disponível em: <https://www.alura.com.br/artigos/como-criar-interface-grafica-swing-java>. Acesso em: 1 set. 2026.

BRASIL. Ministério da Educação. *Programação Orientada a Objetos*. Rede e-Tec Brasil, 2012. Disponível em: https://redeetec.mec.gov.br/images/stories/pdf/eixo_infor_comun/tec_inf/081112_progr_obj.pdf. Acesso em: 1 set. 2026.

MICROSOFT. SDI e MDI. *Microsoft Learn*, 2026. Disponível em: <https://learn.microsoft.com/pt-br/cpp/mfc/sdi-and-mdi?view=msvc-170>. Acesso em: 1 set. 2026.

FORMULÁRIOS em abas. *DevMedia*, 2009. Disponível em: <https://www.devmedia.com.br/artigo-club-delphi-110-formularios-em-abas/14379>. Acesso em: 1 set. 2026.

APLICAÇÕES Desktop no .NET. *DevMedia*, [s. d.]. Disponível em: <https://www.devmedia.com.br/artigo-easy-net-magazine-1-aplicacoes-desktop-no-net/16590>. Acesso em: 1 set. 2026.